RPGXchange

One format. Infinite adventures.

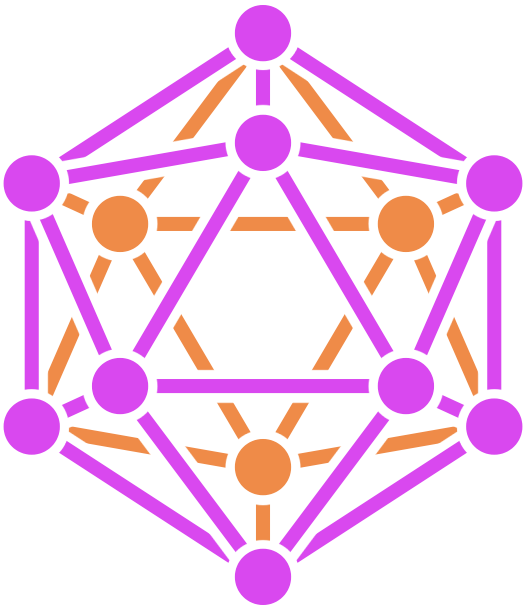


Table of contents:

RPGXchange v0.1.0

File Structure Overview

- Basic Structure
- Assets Directory
- Data Directory
- Best Practices
- Version Control
- File Extensions

Metadata

- Properties
  - Example

Variables

- Location
- Structure
  - Properties
  - Example
- Value Field
  - Type Validation
  - String Interpolation
    - Example
  - Type Coercion
  - Examples
- Best Practices

Namespaces

- Reserved Directory Names
- Namespace Path Specification
- Example Structure:

Formulas

- Basic Syntax
- Operators
- Operator Precedence
- Variable References
- Built-in Functions
- Error Handling

Resource IDs

Versioning
- File Versioning
  - Version Selectors

License
- Branding and Trademark
- Code and Documentation License

RPGXchange v0.1.0

RPGXchange (RPGX) is an open specification for a versatile data interchange format designed to bridge the gap between various tabletop roleplaying game (TTRPG) platforms, tools, and applications. This format enables seamless transfer of character data, game content, and resources across virtual tabletops (VTTs), character managers, digital assistants, and other gaming platforms.

> **WARNING**
>
> This version of RPGXchange is a development version and incomplete. While we encourage exploration and feedback, we recommend against using it in production environments.

File Structure Overview

RPGXchange (RPGX) files are ZIP archives containing structured data and assets for tabletop roleplaying games. This document outlines the standard file structure for both RPGXSource (RPGXS) and RPGXCharacter (RPGXC) files.

Basic Structure

Located within the root of the archive file, is a `metadata.json` file that contains information about the RPGX file. There is also optionally a `data` folder and an `assets` folder to contain the contents of the RPGX file. Some platforms or clients may define additional directories or files for additional functionality.

```
rpgx-file.rpgx/
├─ assets/
├─ data/
└─ metadata.json
```

Assets Directory

The `assets/` directory contains all media files used by the RPGX file. Assets should be specified by their path relative to the assets folder.

Data Directory

The `data/` directory contains all game-related data.

Best Practices

1. Use clear, descriptive namespaces

2. Keep file names short but meaningful
3. Group related content in subdirectories
4. Use platform-specific overrides sparingly
5. Maintain backward compatibility when possible
6. Document any custom implementations
7. Follow JSON schema definitions
8. Include appropriate validation rules
9. Implement proper version control

Version Control

RPGX files follow semantic versioning (MAJOR.MINOR.PATCH) conventions. More specific versioning details can be found on Versioning.

File Extensions

- `.rpgxs`: RPGXSource files
- `.rpgxc`: RPGXCharacter files
- `.rpgx`: Generic RPGXchange files

Metadata

The `metadata.json` file is required in the root directory and contains essential information about the RPGX file.

Properties

Property	Type	Required	Default	Description
id	string	Yes	-	The semi-unique identifier for the RPGX file; Must follow resource id rules
name	string	No	Value of `id`	The user friendly name of the RPGX file
description	string	No	-	The user friendly description of the RPGX file
version	string	No	-	The version of the RPGX file in the format of X.Y.Z with the Y and Z sections being optional; See Versioning
type	string	No	`source` for RPGXS files	The type of RPGX file: either

Property	Type	Required	Default	Description
			and `character` for RPGXC files	`"source"` or `"character"`
namespace	string	No	-	The root namespace of the RPGX file
authors	array	No	`[]`	A list of authors of the project. Can be strings or objects with name, email, url, and/or credit fields
dependencies	array	No	`[]`	A list of required RPGX files. Can be strings or objects with id, and version selector
rpgxVersion	string	No	`undefined`	The target version of RPGX to use. See Versioning

Example

```
{
  "id": "example",
  "name": "Example RPGX File",
  "description": "A comprehensive example of an RPGX file
structure",
  "version": "1.0.0",
```

```
  "type": "source",
  "namespace": "example",
  "authors": [
    {
      "name": "John Doe",
      "email": "john@example.com",
      "url": "https://example.com/johndoe",
      "credit": "Creative design"
    },
    "Jane Smith"
  ],
  "dependencies": [
    {
      "id": "core",
      "version": "1.0"
    },
    "fantasy"
  ],
  "rpgxVersion": "0.1.0"
}
```

Variables

Variables are values that are stored within a RPGX file inside of `variables.json` files located in the ZIP archive. These files must be located within the `data` folder in a directory structure that correspondes to their namespace.

Location

Variables must be organized in `variables.json` files within the `data` directory. The path to the file should match the namespace of the variables. For example:

- Variables in namespace `combat.stats` should be in `/data/combat/stats/variables.json`
- Variables in namespace `character` should be in `/data/character/variables.json`

Structure

Variables are defined as JSON objects with specific properties that determine their behavior and characteristics.

Properties

Property	Type	Required	Default	Description
type	string	No	`any`	The data type of the variable; Must be one of: `integer`, `number`, `string`, `boolean`, `null`, or `any`
description	string	No	-	A user-friendly description of what the variable represents
value	varies	No	`null`	The value of the variable

Example

```
{
    "strength": {
        "type": "integer",
        "description": "The physical strength attribute of the
character",
        "value": 10
    },
    "bonus": {
        "type": "number",
        "description": "Calculated bonus damage based on
strength",
        "value": "{floor(character.stats.strength / 2)}"
    },
    "isAlive": {
        "type": "boolean",
        "description": "Indicates if the character is currently
alive",
        "value": true
    },
    "title": {
        "type": "string",
        "description": "The character's earned title",
        "value": "Novice"
```

```
    }
}
```

Value Field

The `value` field can contain either:

- A literal value matching the specified type
- A formula string enclosed in curly braces `{}`

Type Validation

Values are validated against their declared type:

Type	Valid Values	Examples
`integer`	Whole numbers	`5`, `{floor(3.7)}`
`number`	Any numeric value	`3.14`, `{character.abilities:strength * 1.5}`
`string`	Text strings	`"Hello"`, `{name + " " + title}`
`boolean`	True/false values	`true`, `{health > 0}`
`any`	Any valid JSON value	`42`, `"text"`, `true`
`null`	Null value only	`null`

String Interpolation

String values can include embedded expressions using curly braces `{}`. These expressions are evaluated and their results are inserted into the string.

To include literal curly braces in a string, use double curly braces `{{}}`.

Example

```
{
    "greeting": {
        "type": "string",
        "value": "Hello, {character:name}!"  // Evaluates
character.name
    },
    "status": {
        "type": "string",
        "value": "HP: {{current}} / {maxHealth}"  // Shows "
{current}" literally and evaluates maxHealth
    }
}
```

Type Coercion

When a value doesn't match the declared type:

1. Numbers are rounded to integers for `integer` type
2. Non-boolean values are converted to boolean (`0`, `""`, become `false`; `null` converts to `false` if value is non-nullable)
3. All other values are converted to strings when type is `string` (`undefined` and `null` are converted to empty string)
4. Invalid conversions result in `undefined`

Examples

```
{
    "health": {
        "type": "integer",
        "value": 10.7        // Coerced to 11
    },
    "name": {
        "type": "string",
        "value": 42          // Coerced to "42"
    },
```

```
    "active": {
        "type": "boolean",
        "value": 1           // Coerced to true
    }
}
```

Best Practices

1. Use descriptive variable names
2. Keep namespaces organized
3. Consider variable dependencies
4. Use type coercion sparingly

Namespaces

Namespaces in RPGX files are determined by the resource path within rooted directories (e.g., `data` or `assets`) inside the ZIP archive. The namespace hierarchy follows folder paths, with each subdirectory representing a segment of the namespace. Every namespace is prepended by the namespace specified in the metadata.json for the RPGX file.

Reserved Directory Names

Some folder names are reserved and do not contribute to namespace paths. These folders are designated for containing files of specific functionality and purpose.

Namespace Path Specification

Namespaces are specified using the path with dots (`.`) as segment separators. When referencing a namespace, the system will match to the closest matching namespace segment that aligns with the provided path. For instance, if referencing `combat.spells`, it will match the nearest namespace that ends with these segments.

To avoid ambiguity in namespace resolution, it's recommended to use absolute paths that include all namespace segments from the root. This ensures precise and predictable namespace matching.

Example Structure:

```
/                   # Base namespace from metadata.json:
example
└─ data/            # Reserved directory
   ├─ combat/       # Namespace: example.combat
```

```
    │  ├─ spells/    # Namespace: example.combat.spells
    │  └─ weapons/   # Namespace: example.combat.weapons
    └─ characters/   # Namespace: example.characters
```

Formulas

Formulas in RPGX allow dynamic computation of values. They can be used in variables, conditions, and other dynamic fields.

Basic Syntax

In non-dedicated fields, formulas are enclosed in curly braces. In fields expecting a formula, enclosing curly braces are not necessary.

Operators

Operator	Description	Example	Notes
`+`	Addition/concatenation	`1 + 2`, `"a" + "b"`	When either operand is a string, the other operand will be converted into a string; else, both values are converted to numbers when possible; otherwise, results in `undefined`
`-`	Subtraction	`5 - 3`	Both operands are converted to numbers when possible; otherwise, results in `undefined`
`*`	Multiplication	`4 * 2`	Both operands are converted to

Operator	Description	Example	Notes
			numbers when possible; otherwise, results in `undefined`
`/`	Division	`10 / 2`	Both operands are converted to numbers when possible; otherwise, results in `undefined`
`%`	Modulo	`7 % 3`	Both operands are converted to numbers when possible; otherwise, results in `undefined`
`==`	Equality	`x == y`	
`!=`	Inequality	`x != y`	
`>`	Greater than	`x > y`	If either value is not comparable, then `undefined` is returned
`>=`	Greater or equal	`x >= y`	If either value is not comparable, then `undefined` is returned
`<`	Less than	`x < y`	If either value is not comparable, then `undefined` is returned

Operator	Description	Example	Notes
`<=`	Less or equal	`x <= y`	If either value is not comparable, then `undefined` is returned
`&&`	Logical AND	`x > 0 && y < 10`	Both operands are coerced into boolean values
`			`	`	Logical OR

Operator Precedence

Operators are evaluated in the following order (highest to lowest):

1. Parentheses `()`
2. Multiplication/Division/Modulo `* / %`
3. Addition/Subtraction `+ -`
4. Comparison `< <= > >=`
5. Equality `== !=`
6. Logical AND `&&`
7. Logical OR `||`

Variable References

Variables and other resources can be referenced using either its id or namespace and id. When referencing a variable with its namespace, use the namespace path followed by a colon followed by the resource id:

- Resource Id: `strength`
- Namespaced reference: `character.stats:strength`

Built-in Functions

Function	Description	Example
`floor(x)`	Round down	`floor(3.7)` -> `3`
`ceil(x)`	Round up	`ceil(3.2)` -> `4`
`round(x)`	Round to nearest	`round(3.5)` -> `4`
`min(x,y)`	Minimum value	`min(5, 3)` -> `3`
`max(x,y)`	Maximum value	`max(5, 3)` -> `5`
`if(c,t,f)`	Conditional	`if(x>0, x, 0)`
`length(x)`	Gets the length of a string	`length("xyz")` -> 3

Error Handling

Any time an error occurs, `undefined` can be used to continue calculation. Scenarios that may warrant use of `undefined` includbut are not limited to the following cases:

- Division by zero
- Invalid type conversions
- Missing variable references
- Invalid function arguments
- Comparison of incompatible types

Resource IDs

Resource IDs are semi-unique identifiers used for file IDs in the metadata, namespaces, and other resources. Resource IDs must adhere to the following rules:

- Must be alphanumeric (both uppercase and lowercase letters are allowed).
- Can contain underscores.
- Cannot start with a number.
- Cannot start or end with an underscore.

Versioning

File Versioning

RPGX files should include a version number that follows Semantic Versioning (SemVer) principles. The version number should be specified in the format `MAJOR.MINOR.PATCH`:

- `MAJOR` version changes indicate incompatible modifications
- `MINOR` version changes add functionality in a backward-compatible manner
- `PATCH` version changes include backward-compatible bug fixes

For example, an RPGX file at version `1.2.3` indicates:

- Major version: 1
- Minor version: 2
- Patch version: 3

We strongly recommend using SemVer to ensure consistent version management and compatibility checking across RPGX files.

Version Selectors

Version selectors follow semantic versioning (SemVer) rules and can be expressed in several ways:

Selector	Description	Example
`=x.y.z`	Exact version match	`=1.2.3` matches only 1.2.3
`>x.y.z`	Greater than version	`>1.2.3` matches 1.2.4, 1.3.0, 2.0.0

Selector	Description	Example
`>=x.y.z`	Greater than or equal to	`>=1.2.3` matches 1.2.3, 1.2.4, 1.3.0
`<x.y.z`	Less than version	`<1.2.3` matches 1.2.2, 1.1.0
`<=x.y.z`	Less than or equal to	`<=1.2.3` matches 1.2.3, 1.2.2, 1.1.0
`x.y.z`	Exact version (= implied)	`1.2.3` matches only 1.2.3

Multiple selectors can be combined:

- Using spaces for AND: `>=1.2.0 <2.0.0`
- Using `||` for OR: `1.2.3 || >=1.3.0 <2.0.0`

Prerelease versions (e.g., `1.2.3-alpha.1`) are only included when explicitly specified in the version selector.

License

Branding and Trademark

The RPGXchange logo, favicon, and branding assets are **NOT** covered by the CC BY-SA 4.0 license. These assets are protected intellectual property.

See the Branding Guidelines for complete trademark usage terms.

Quick summary:

- ✅ Use in documentation to show compatibility with the format
- ✅ Use badges/logos to indicate format support
- ❌ Do not modify the logo
- ❌ Do not use in derivative projects claiming to be RPGXchange
- ❌ Do not use in ways suggesting ownership or endorsement

Full branding guidelines

Code and Documentation License

All code and documentation in this repository are licensed under the **Creative Commons Attribution-ShareAlike 4.0 International** license.

This means you can:

- Use, modify, and distribute the code and documentation
- Create derivative works
- Use for commercial purposes

Provided that you:

- Give appropriate credit to RPGXchange
- License your derivative works under the same license

- Indicate what changes you made

For the complete legal text, see the LICENSE file in the repository root.